

禾书耕文（GrainScript） — 软件架构与设计模式说明

用途：供软件工程 / 软件设计课程展示：系统边界、子系统划分、分层结构、主要设计模式。
读者：模式设计课老师、评审同学（无需阅读源码即可把握整体）。
版本：2026-06-01 · 与 docs/ARCHITECTURE.md 、 docs/WORKFLOWS.md 互补。

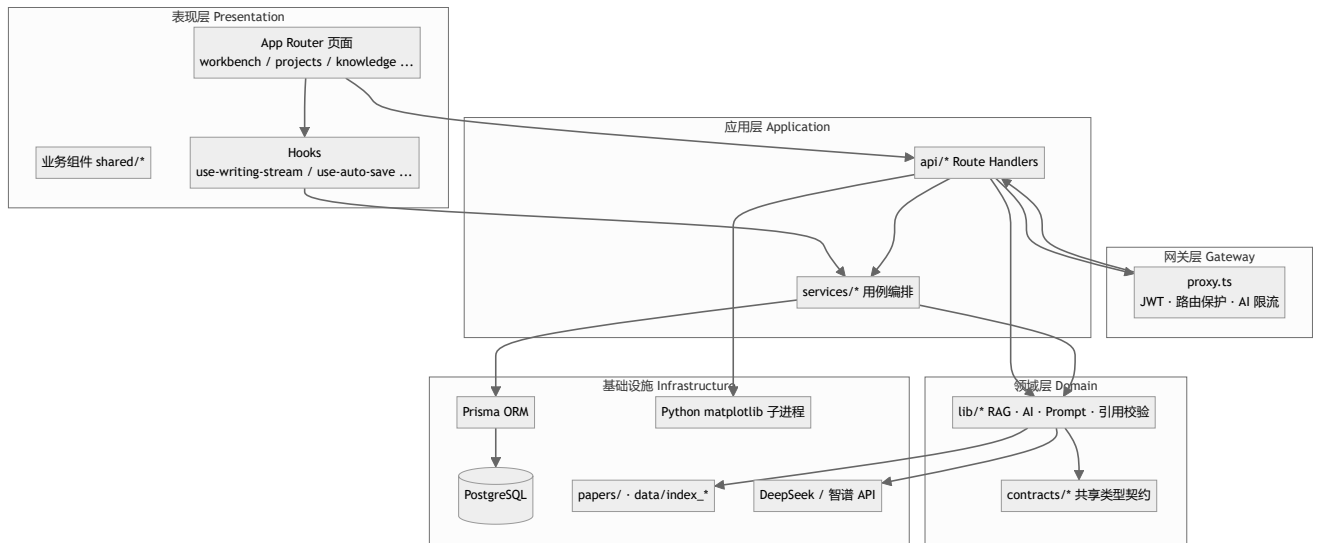
1. 系统概述

项	说明
名称	禾书耕文（GrainScript）
类型	B/S 架构的 Web 应用（Next.js 全栈单体）
领域	农业 / 材料等科研场景的 SCI 论文辅助写作
核心能力	私有文献 RAG、多 Agent 扩写、引用核查、查重降重、图表与 XRD 分析、多格式导出
部署形态	单实例 Node 服务 + PostgreSQL + 本地/磁盘文献与向量索引

一句话架构：浏览器 → 网关层（认证/限流） → 表现层（React 页面 + Hooks） → 应用 API（Route Handlers） → 领域服务与 Lib → PostgreSQL / 文件系统 / 外部 LLM API / Python 子进程。

2. 逻辑分层（Layered Architecture）

采用经典**四层 + 契约层**，依赖方向自上而下，禁止 UI 直接 fetch 裸调（规范要求走 services/ ）。



层	目录	职责
契约层	src/contracts/	前后端共享的 TypeScript 类型（SSE、Project、Writing、Figure 等），相当于 DTO / 领域词汇表

层	目录	职责
表现层	src/app/*、src/components/、src/hooks/	用户交互、状态、流式 UI (SSE 解析)
网关层	src/proxy.ts	横切关注点：登录校验、x-user-id 注入、AI 接口按用户限流
应用层	src/app/api/、src/services/	HTTP 入口、用例组合 (如查重流水线、证据包构建)
领域层	src/lib/	与框架无关的核心算法：RAG、多 Agent Prompt、引用验证
基础设施	prisma/、data/、scripts/	持久化、索引文件、离线建索引与绘图

3. 子系统划分 (Subsystem Catalog)

系统按**业务域**拆为 12 个子系统；多数通过 `src/lib/module-registry.ts` 的 **模块注册表** 在首页/工作台侧栏统一挂载 (配合 **功能开关**)。

编号	子系统	用户可见入口	核心后端	持久化
S1	身份与访问控制	登录 / 注册	proxy.ts、lib/auth.ts、api/auth/*	User
S2	项目管理	项目列表、工作台	api/projects/*、services/project.ts	Project、Section、Reference
S3	AI 写作管道	工作台扩写	api/writing、lib/ai.ts、lib/prompts/*	章节增量写入 Section
S4	大纲生成	大纲页	api/outline	Project.outline
S5	知识库与 RAG	文献库、文献对话	lib/rag.ts、api/knowledge/*、api/chat	KnowledgeFile、KnowledgeChunk + data/index_*
S6	引用与文献管理	参考文献面板	lib/citation*.ts、api/references	Reference、ReferenceSource
S7	数据分析与证据	分析页、工作台数据注入	api/analysis、services/evidence-pack.ts	AnalysisResult、Project.dataClaims
S8	质量：查重与降重	查重页	api/plagiarism/*、services/plagiarism-check.ts	PlagiarismCheck、PlagiarismMatch、RewriteSuggestion
S9	质量：一致性 / 审查	工作台对话框、审查中心	api/consistency/*、api/review/*	ReviewCheck 等

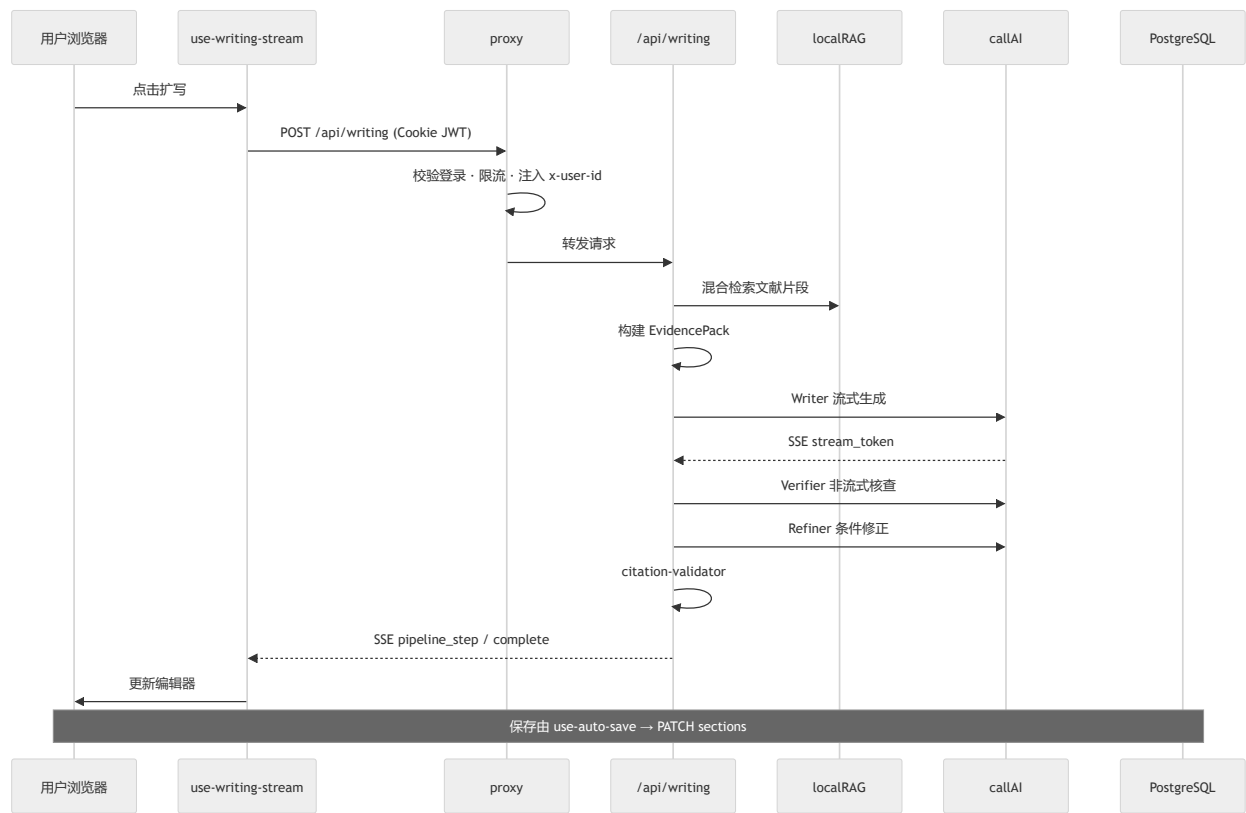
编号	子系统	用户可见入口	核心后端	持久化
S10	图表与XRD	绘图页、XRD实验室	api/chart 、 api/figures/registry 、 scripts/charts/	项目 charts JSON、图片文件
S11	导出与阅读	PDF 阅读器、导出	api/export/* 、 api/pdf 、 services/server-pdf.ts	临时文件 / 流式响应
S12	管理后台	/admin	api/admin/* 、 lib/admin-auth.ts	全库只读/运维操作

横切子系统（Cross-Cutting）：

- 配置与密钥：lib/settings.ts （Admin 加密存 API Key，运行时热加载）
- 可观测性：lib/usage-log.ts （内存环形缓冲 AI 调用日志）
- 功能开关：lib/feature-flags.ts （环境变量控制模块可见性）

4. 子系统协作图（核心路径）

4.1 写作主路径（S2 + S3 + S5 + S6）



4.2 多用户边界（S1 + S2）

- 已隔离：Project.userId，项目 CRUD 带 where: { userId }。

- **实验室共享**: S5 知识库与 RAG 索引为全站一份（无 per-user 文献库）。
- **并发**: PostgreSQL 支持多连接; AI 限流按用户约 10 次/分钟（内存 Map, 单进程有效）。

5. 设计模式对照表（课程重点）

下表将代码中的惯用法映射到经典 GoF / 架构模式名称，便于答辩表述。

模式	在本系统中的体现	典型位置
分层架构 (Layered)	Presentation → API → Service → Lib → DB	见第 2 节
管道 / 流水线 (Pipeline)	Writer → Verifier → Refiner → 引用/数据校验, 阶段 SSE 上报	api/writing/route.ts 、 docs/WORKFLOWS.md
策略 (Strategy)	多 LLM 厂商/角色配置; 查重降重多种 strategy ; 引用格式 citationStyle	lib/models.ts 、 getAgentModelConfig 、 查重 UI
模板方法 (Template Method)	Prompt 构建步骤固定, 子模块填充领域片段	lib/prompts/*.ts
注册表 (Registry)	功能模块、图表类型统一登记, 新增条目即可扩展	module-registry.ts 、 scripts/charts/registry.json
工厂方法 (Factory Method)	Python ChartModule 子类实现 plot() , 基类定义算法骨架	scripts/charts/chart_base.py
单例 (Singleton)	进程内唯一 RAG 引擎、用量日志缓冲	export const localRAG 、 usage-log.ts
外观 (Facade)	统一 AI 调用入口, 隐藏重试、计费、流式解析	lib/ai.ts callAI / streamAIResponse
仓储 (Repository)	Prisma Client 封装 CRUD, 屏蔽 SQL	lib/prisma.ts + schema.prisma
DTO / 值对象契约	请求响应类型与 SSE 联合类型	src/contracts/*
判别联合 + 类型守卫 (Tagged Union)	SSE 事件 { type, ... } 分支解析	contracts/sse.ts 、 前端 stream 解析
观察者 / 发布订阅 (Observer)	服务端 SSE 推送写作进度, 前端订阅更新 UI	use-writing-stream.ts
责任链 (Chain of Responsibility)	请求先经 proxy: 认证 → 限流 → 再进入 Route	src/proxy.ts
装饰 / 横切 (Cross-Cutting)	认证头注入、Admin requireAdmin 、 Zod validateBody	proxy.ts 、 admin-auth.ts 、 api-validate.ts
适配器 (Adapter)	Node 调 Python 子进程生成图表; Playwright 导出 PDF	api/chart/route.ts 、 api/export/pdf

模式	在本系统中的体现	典型位置
BFF (Backend for Frontend)	Next.js Route Handler 为前端定制聚合 API	全部 <code>src/app/api/*</code>
功能开关 (Feature Toggle)	环境变量控制模块是否展示	<code>feature-flags.ts</code> + <code>APP_MODULES</code>
降级 (Graceful Degradation)	RAG 无结果仍写作; Verifier 模型不可用换 DeepSeek	<code>writing/route.ts</code> 、 <code>models.ts</code>

说明：项目未引入重型 DI 框架；依赖通过 **显式 import + 纯函数/单例导出** 完成，符合单人全栈、KISS 原则。

6. 关键子系统设计要点

6.1 S3 — 多 Agent 写作管道 (Pipeline + Strategy)

问题：单模型易幻觉引用且无法自审。
结构：三角色链式处理 + 后置规则引擎。

阶段	角色	模式要点
检索	RAG	混合检索，为引用提供可追溯片段
生成	Writer	策略：默认 DeepSeek，流式 SSE
核查	Verifier	策略：异厂商模型（智谱），独立验证
修正	Refiner	条件触发，非总是执行
校验	citation-validator	确定性规则，非 LLM

6.2 S5 — RAG 子系统 (Composite Retrieval)



- **EmbeddingStore**: 向量与元数据分离存储（`.emb` 二进制 + JSON 元数据）。
- **localRAG 单例**: 避免重复加载大索引。

6.3 S10 — 图表子系统 (Registry + Template Method)

- **注册表**: `registry.json` 为唯一真相源，前端 `GET /api/figures/registry` 动态渲染。
- **扩展方式**: 新增 `chart_types/xxx.py` 继承 `ChartModule` + 注册表一条记录（**开闭原则**）。

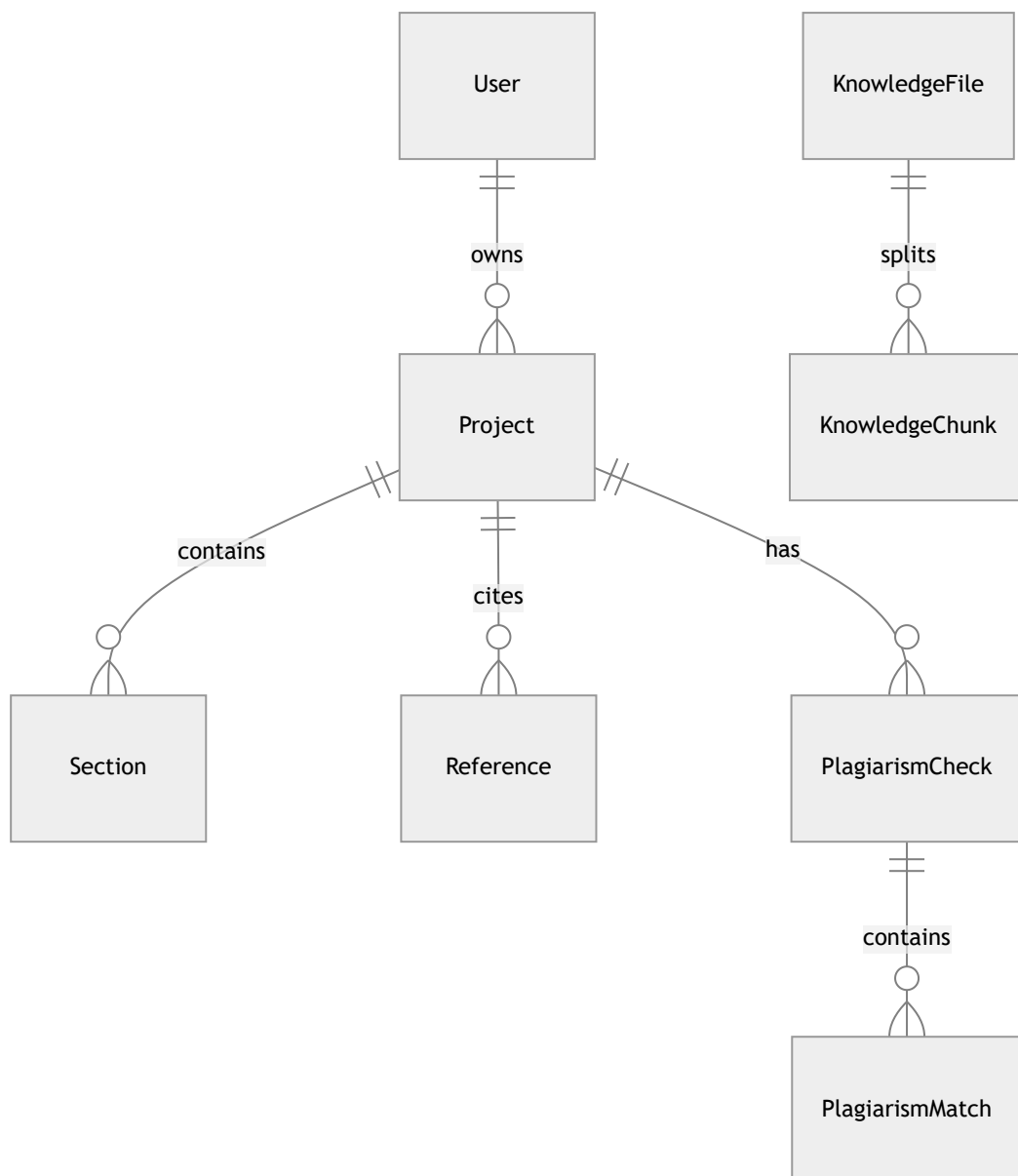
6.4 S1 — 安全网关 (Chain of Responsibility)

proxy.ts 统一处理：

1. JWT 解析 → `userId`
2. 保护路由表 `protectedPages` / `protectedApis`
3. AI 路径 `checkRateLimit(userId || IP)`

Route Handler 不重复写登录逻辑，符合 单一职责。

7. 数据架构（简图）



- 聚合根： `Project` （论文项目生命周期中心）。
- 共享数据： `KnowledgeFile` （无 `userId`，实验室级文献池）。

8. 技术栈与子系统映射

技术	服务的子系统
Next.js 16 App Router	全部 UI + BFF API
React 19 + Hooks	S2、S3 状态与 SSE 消费
TipTap	S2 工作台编辑器
Prisma + PostgreSQL	S1、S2、S8、S9 持久化
jose + bcrypt	S1 认证
自研 fetch AI 层	S3、S4、S5 对话、S7
自研 RAG	S5
Python matplotlib	S10
Playwright	S11 PDF 导出
Vitest	回归测试 (lib/utils、契约等)

9. 质量属性 (Architecture Trade-offs)

属性	设计选择	代价
可维护性	契约层 + services 分离	大文件仍待拆 (workbench、writing-panel)
可扩展性	模块/图表注册表	新图表需维护 Python 环境
性能	本地索引 + 内存缓存 RAG	单进程内存占用高；多实例需共享索引策略
安全	JWT + 项目 userId 隔离	知识库共享；部分 API 未校验 project 归属
可用性	SSE 进度、Pipeline 分步反馈	长连接占用；AI 失败需前端处理 error 事件

10. 推荐阅读顺序 (给老师 5 分钟版)

- 本文 §2 分层 + §3 子系统表
- docs/ARCHITECTURE.md — 三个核心架构决策 (多 Agent、RAG、引用验证)
- docs/WORKFLOWS.md — 三条端到端流程 (写作 / 保存 / 查重)
- 可选: docs/CODE_MAP.md — 功能 → 文件索引

11. 答辩可用「设计亮点」三句话

1. **业务上**：用「多 Agent 异构审查 + RAG 可追溯引用」解决科研写作里最常见的幻觉引用问题，而不是单 Prompt 生成。
 2. **架构上**：Next.js 单体 + 清晰契约层与服务层，网关横切认证限流，适合实验室小规模多用户。
 3. **工程上**：注册表驱动模块与图表扩展、SSE 管道可观测、混合检索兼顾中英文术语——体现**可扩展性与领域适配**的平衡。
-

文档维护：子系统增删或模式重构时，请同步更新 \$3 与 \$5。